competitive-programming · lib/geometry

Geometría

Manual de consulta de los 21 headers

Introducción	índice, dependencias, `db` o `ll`
1	Primitives — punto, orientación, distancias, intersecciones
2	Polygons — hull, área, punto en polígono, Minkowski, semiplanos
3	Circles — intersección, tangentes, circuncentro, círculo mínimo
4	Misc — par más cercano, MST Manhattan, colineales

Generado el 2026-11-03 desde el commit `1f1f79a`

23 figuras · los problemas de práctica están aparte, en `PROBLEMS.md`

Contenido

Geometría 4

- Headers 4
- Dependencias 5
- T: db o ll 5
- Cómo se pega 5

Primitives 7

- Cómo se arma la submission 7
- Contrato de tipos 8
- Point.h 9
- AngleCmp.h 12
- SegDist.h 14
- SegIsect.h 15
- Casos degenerados 17
- Ejemplo completo 18
- Verificación 18

Polygons 19

- Cómo se arma la submission 19
- Dos convenciones 20
- ConvexHull.h 20
- ConvexHullGraham.h 22
- PolygonCenArea.h 22
- InPolygon.h 23
- HullDiameter.h 24
- HullTangents.h 24
- MinkowskiSum.h 25
- LineHull.h 26
- HalfPlaneIsect.h 27
- Casos degenerados 28
- Ejemplo completo 29
- Verificación 30
- Lo que no está 30

Circles 31

- Cómo se arma la submission 31
- Contrato de tipos 32
- Circle.h 32
- CircleIsect.h 33
- CircleTangents.h 34
- Circumcenter.h 35
- MinEnclosingCirc.h 36
- Casos degenerados 37
- Ejemplo completo 38
- Verificación 39

Misc **40**
Cómo se arma la submission 40
Choques de nombres 40
ClosestPair.h 41
ManhattanMST.h 41
MaxCollinear.h 42
Casos degenerados 43
Ejemplo completo 43
Verificación 44

Geometría

21 headers, 4 manuales, 23 figuras. Compilan juntos sin warnings; ~1.280.000 casos verificados.

Los manuales documentan **solo los métodos**: qué hace cada uno, sus restricciones, hasta dónde aguanta y su complejidad. Dónde practicar está en **PROBLEMS.md**.

Headers · Dependencias · `T` : `db` o `ll` · Cómo se pega · Problemas

directorio	qué cubre	headers
Primitives	punto, orientación, distancias, intersección, orden angular	4
Polygons	hull, área, punto en polígono, tangentes, Minkowski, semiplanos	9
Circles	intersección, tangentes, circuncentro, círculo mínimo	5
Misc	par más cercano, MST Manhattan, máximos colineales	3

Empezá por **Primitives**. Todo lo demás supone `Point.h`.

Headers

Primitives

header	qué da	
`Point.h`	el tipo `P` y 14 operaciones (`cross` , `dot` , `sgn` , `foot` , `onSeg` , …)	$O(1)$
`AngleCmp.h`	orden antihorario, sin `atan2`	$O(1)$
`SegDist.h`	distancia punto–recta y punto–segmento	$O(1)$
`SegIsect.h`	intersección recta/segmento	$O(1)$

Polygons

header	qué da	
`ConvexHull.h`	hull por cadena monótona, + `ulHull`	$O(N \log N)$
`ConvexHullGraham.h`	hull por Graham — *alternativa*, no complemento	$O(N \log N)$
`PolygonCenArea.h`	centroide y área con signo	$O(N)$
`InPolygon.h`	adentro / borde / afuera	$O(N)$
`HullDiameter.h`	máxima distancia al cuadrado	$O(N \log N)$
`HullTangents.h`	tangentes desde un punto exterior	$O(\log N)$
`MinkowskiSum.h`	suma de dos convexos	$O(N)$
`LineHull.h`	recta contra convexo, vértice extremo	$O(\log N)$
`HalfPlaneIsect.h`	intersección de semiplanos	$O(N \log N)$

Circles

header	qué da	
`Circle.h`	tipo `Circ` , `in` , `arcLength`	$O(1)$

header	qué da	
`CircleIsect.h`	círculo–círculo, círculo–recta, área común	$O(1)$
`CircleTangents.h`	tangentes internas y externas	$O(1)$
`Circumcenter.h`	circuncentro y circunradio	$O(1)$
`MinEnclosingCirc.h`	círculo mínimo	$O(N)$ esperado

Misc

header	qué da	
`ClosestPair.h`	los dos puntos más cercanos	$O(N \log N)$
`ManhattanMST.h`	$4N$ aristas que contienen un MST	$O(N \log N)$
`MaxCollinear.h`	máximos puntos sobre una recta	$O(n^2 \log n)$

Dependencias

Todos piden `Point.h` y nada más, salvo:

- `HalfPlaneIsect.h` → + **`AngleCmp.h`**
- `HullDiameter.h` → + **un hull** (llama a `hull()`; sirve cualquiera)
- `CircleIsect.h`, `CircleTangents.h`, `Circumcenter.h` → + **`Circle.h`**
- `MinEnclosingCirc.h` → + **`Circle.h`**, **`Circumcenter.h`**
- `ManhattanMST.h` → **nada** (trabaja con `vpi`; después corré Kruskal + `DSU`)

> `ConvexHull.h` y `ConvexHullGraham.h` definen los dos `hull` e `hullInd`: **son alternativas, elegí una.** Usá `ConvexHull.h`.

`T`: `db` o `ll`

	`db` (default)	`ll`
`sgn`, `cross`	con `EPS`	**exactos**
`abs`, `arg`, `unit`, `foot`	correctos	**truncan**

Coordenadas enteras y solo orientación, área o comparar distancias → `ll`. Distancias o ángulos reales → `db`.

> **Nunca** `double`. `long double` hace exacto el `cross` de 3 puntos hasta $|c| \approx 2{,}1 \cdot 10^9$; `double` falla desde $5 \cdot 10^7$. — detalle

Cómo se pega

`templateHeavy.cpp` → `E.cpp`, y pegás el **código** del header en la sección de template, `Point.h` primero.

> **Los `#include` no van en la solución.** Ningún header tiene uno. El campo `Requiere:` solo indica qué más pegar y en qué orden.

Choques de nombres → Misc · Verificación por rutina → final de cada manual · Lo que quedó afuera de cp-notebook → Polygons

Primitives

El tipo punto y todo lo que se construye sobre él. `Circles`, `Polygons` y `Misc` se apoyan en `Point.h`.

Manual de consulta: **qué hace cada método, qué le podés pasar y hasta dónde aguanta.** Los problemas donde practicar están en PROBLEMS.md.

Índice

- Cómo se arma la submission
- Contrato de tipos
  - Elegir `T`: `db` o `ll`
  - Hasta dónde aguanta `db`
  - Puntos calculados: el límite del `EPS`
- `Point.h`
  - Operadores
  - `sgn`, `sq`
  - `abs2`, `abs`
  - `arg`, `dir`
  - `conj`, `perp`, `unit`
  - `dot`
  - `cross`
  - `reflect`, `foot`
  - `onSeg`
- `AngleCmp.h`
  - `half`
  - `angleCmp`
- `SegDist.h`
  - `lineDist`
  - `segDist`
- `SegIsect.h`
  - `lineIsect`
  - `strictIsect`
  - `segIsect`
- Casos degenerados
- Ejemplo completo
- Verificación

Cómo se arma la submission

Copiás `templateHeavy.cpp` en `E.cpp` y pegás el **código** del header en la sección de template.

> **Los `#include` no van en la solución.** El campo `Requiere:` es solo un indicador de qué más pegar, y en qué orden.

si pegás	pegá antes
`AngleCmp.h`, `SegDist.h`, `SegIsect.h`	`Point.h`

Contrato de tipos

```
using T = db;                   // el escalar: db = long double, o ll
const T EPS = 1e-9;
using P = pair<T,T>;            // un punto, o un vector
using vP = V<P>;
using Line = pair<P,P>;         // dos puntos: una recta, o un segmento
```

`P` es a la vez **punto** y **vector**. `Line` es a la vez **recta infinita** y **segmento** — cuál de las dos lo decide la función que lo recibe, y está dicho en cada una.

Elegir `T`: `db` o `ll`

	`T = db` (default)	`T = ll`
predicados (`sgn`, `cross`, `dot`)	con `EPS`	**exactos**
métrica (`abs`, `arg`, `unit`, `foot`, `reflect`)	correctos	**truncan en silencio**

Con `T = ll`, `EPS = 1e-9` trunca a `0` y `sgn` se vuelve exacto — eso es deseable. Pero `abs({1,1})` devuelve `1` en vez de $\sqrt{2}$, y `arg` devuelve `0` casi siempre.

Regla: coordenadas enteras y solo orientación, área o comparar distancias → `T = ll`, sin usar `abs` / `arg` / `unit`. Distancias o ángulos reales → `T = db`.

Hasta dónde aguanta `db`

En todo el manual, C = **la mayor coordenada en valor absoluto** de los puntos de entrada: $|c| \le C$.

Cuánto valen los productos. Verificado por fuerza bruta sobre todas las ternas de la grilla $[-C, C]^2$:

expresión	máximo	con `ll` entra hasta
`cross(p,a,b)`	$4C^2$	$C \approx 1{,}5 \cdot 10^9$
`dot(p,a,b)`	$8C^2$	$C \approx 1{,}07 \cdot 10^9$
`abs2(a-b)`	$8C^2$	$C \approx 1{,}07 \cdot 10^9$
`cross(a,b)` de vectores con componentes $\le C$	$2C^2$	$C \approx 2{,}1 \cdot 10^9$

Con `ll` los predicados son **exactos** hasta esos límites. El típico $C = 10^9$ entra en todos.

Con flotantes el problema no es el desborde sino la cancelación. Peor caso real: `a` y `b` casi colineales con `p`, `cross` verdadero entre 1 y 4, coordenadas de magnitud C. 20.000 casos por celda, signo contra `__int128`:

C	`long double` (64 bits de mantisa)	`double` (53 bits)
$2 \cdot 10^7$	0% mal	0% mal
$5 \cdot 10^7$	0% mal	**2,6% mal**
10^8	0% mal	**41,3% mal**

C	`long double` (64 bits de mantisa)	`double` (53 bits)
10^9	**0% mal**	inservible
$3 \cdot 10^9$	14,3% mal	inservible
$5 \cdot 10^9$	50,6% mal	inservible

Umbral por búsqueda binaria: `long double` **exacto hasta** $C \approx 2{,}1 \cdot 10^9$.

> **No cambies** `db` **a** `double` **si usás geometría.** `templateHeavy.cpp` dice "*or double if tight TL*"; con `double` los predicados fallan desde $5 \cdot 10^7$, que es un rango de problema completamente normal.

Qué tipo de error es: en esta familia de peor caso, **todos** los errores medidos fueron "el `cross` colapsa a 0" (un caso casi degenerado se lee como colineal). **Ninguno** invirtió el signo, en 240.000 casos entre las dos precisiones.

Puntos calculados: el límite del `EPS`

Todo lo anterior vale para puntos **de entrada**. Un punto **calculado** — la salida de `lineIsect`, `foot`, `reflect`, `isect` — trae error de redondeo, y ahí aparece otro límite: `EPS = 1e-9` **es absoluto**, mientras que el `cross` crece como C^2.

Medido: `x = strictIsect(a, b)` está, en teoría, sobre los dos segmentos. ¿Cuántas veces `onSeg(x, a) && onSeg(x, b)` dice que **no**?

C	`onSeg` dice `false`
10^3	0%
10^4	0%
10^5	**0,5%**
10^6	**79,9%**
10^9	**87,7%**

Regla: `sgn` **y** `onSeg` **sobre puntos calculados, solo hasta** $C \approx 10^4$. Por encima, decidí con la entrada original (por ejemplo, con el parámetro sobre el segmento) en vez de preguntarle a `onSeg`.

`in()` de `Circle.h` **no** tiene este problema — compara distancias, que escalan con C y no con C^2: medido 0% de error hasta $C = 10^7$ con puntos calculados.

`Point.h`

Operadores

```
P  operator+(P l, P r);      P  operator-(P l, P r);
P  operator*(P l, T r);      P  operator*(T l, P r);     P operator/(P l, T r);
P  operator*(P l, P r);      P  operator/(P l, P r);      // complejos
P& operator+=(P&, const P&); P& operator-=(P&, const P&);
P& operator*=(P&, const T&); P& operator/=(P&, const T&);
```

Suma y resta de vectores; escalado por un escalar (el escalar puede ir de cualquier lado).

`P * P` **y** `P / P` **son multiplicación y división de complejos**, no producto componente a componente:

- `a * b` **rota** `a` por el ángulo de `b` y **escala** por `|b|`.
- `a / b` da el cociente: su ángulo es el **ángulo orientado** de `b` a `a`.

Restricciones: `operator/(P,T)` con `r = 0` y `operator/(P,P)` con `r = (0,0)` dan `nan` · **Tipo**: cualquiera; `P*P` y `P/P` piden `db` · **Aguanta**: `+` y `-` llegan a $2C$; `P*P` multiplica coordenadas, con `ll` llega a $2C^2$ · **Complejidad**: $O(1)$.

`sgn`, `sq`

```
int sgn(T a);    // -1, 0, 1 con tolerancia EPS
T   sq(T a);     // a*a
```

`sgn` es **el único lugar donde entra** `EPS`. Todo lo que compare contra cero debería pasar por acá en vez de usar `== 0`.

Restricciones: ninguna · **Tipo**: cualquiera · **Aguanta**: `EPS` **es absoluto**: con puntos de entrada enteros no molesta, pero sobre puntos **calculados** solo tolera hasta $C \approx 10^4$ (ver tabla); `sq` con `ll` desborda para $|a| > 3 \cdot 10^9$ · **Complejidad**: $O(1)$.

`abs2`, `abs`

```
T abs2(P p);   // x^2 + y^2
T abs(P p);    // sqrt(abs2(p))
```

Para comparar distancias usá `abs2`: evita la raíz y, con `T = ll`, es exacto.

Restricciones: ninguna · **Tipo**: `abs2` cualquiera (con `ll` es exacto); `abs` pide `db` — con `ll` trunca · **Aguanta**: `abs2(a-b)` llega a $8C^2$: con `ll` entra hasta $C \approx 1{,}07 \cdot 10^9$ · **Complejidad**: $O(1)$.

`arg`, `dir`

```
T arg(P p);     // atan2(p.s, p.f), rango (-pi, pi]
P dir(T ang);   // (cos ang, sin ang), el unitario de ese angulo
```

Restricciones: `arg({0,0})` devuelve `0` (no `nan`), pero el origen no tiene ángulo: tratalo aparte · **Tipo**: `db` · **Aguanta**: error relativo $\sim 10^{-18}$, no depende de C · **Complejidad**: $O(1)$.

> **Para ordenar por ángulo no uses** `arg`: usá `angleCmp`. Es más rápido y exacto.

`conj`, `perp`, `unit`

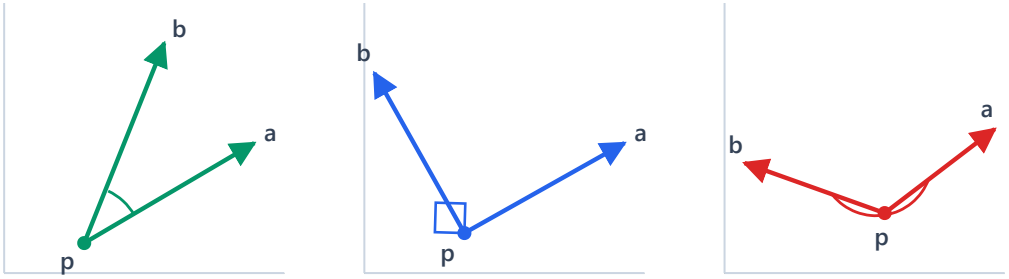
```
P conj(P p);   // (x, -y)        refleja sobre el eje x
P perp(P p);   // (-y, x)        rota 90 grados antihorario
P unit(P p);   // p / abs(p)     mismo angulo, largo 1
```

Restricciones: `unit({0,0})` **devuelve** `(nan, nan)` — divide por cero sin avisar · **Tipo**: `conj` y `perp` cualquiera; `unit` pide `db` · **Aguanta**: `conj` y `perp` no multiplican: nunca desbordan · **Complejidad**: $O(1)$.

`dot`

```
T dot(P a, P b);          // producto escalar
T dot(P p, P a, P b);     // dot(a-p, b-p): el angulo EN p
```

Responde "*¿hacia adelante o hacia atrás?*":



dot(p,a,b) > 0
angulo AGUDO en p

dot(p,a,b) == 0
PERPENDICULARES

dot(p,a,b) < 0
angulo OBTUSO en p

Figura 1. el signo del dot

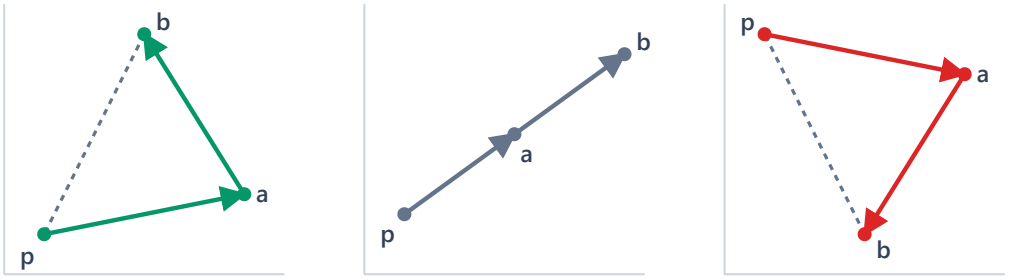
`sgn(dot(p,a,b))`	el ángulo en `p`
`> 0`	agudo
`0`	recto
`< 0`	obtuso

Restricciones: ninguna · **Tipo**: cualquiera; con `ll` es exacto · **Aguanta**: `dot(p,a,b)` llega a $8C^2$: con `ll` entra hasta $C \approx 1{,}07 \cdot 10^9$; con `long double` el signo es exacto hasta $C \approx 2{,}1 \cdot 10^9$ · **Complejidad**: $O(1)$.

`cross`

```
T cross(P a, P b);         // a.x*b.y - a.y*b.x
T cross(P p, P a, P b);    // cross(a-p, b-p): giro p -> a -> b
```

La función más importante de la librería. Es el **área con signo** del paralelogramo, y responde "*¿de qué lado?*":



cross(p,a,b) > 0
gira a la IZQUIERDA (CCW)

cross(p,a,b) == 0
los tres son COLINEALES

cross(p,a,b) < 0
gira a la DERECHA (CW)

Figura 2. el signo del cross da la orientación

`sgn(cross(p,a,b))`	significa
`> 0`	giro **antihorario**: `b` a la izquierda de `p→a`
`0`	**colineales**
`< 0`	giro **horario**: `b` a la derecha

También: $|\text{cross}(p, a, b)|$ es **el doble del área** del triángulo `pab`.

Restricciones: ninguna · **Tipo**: cualquiera; con `ll` es exacto · **Aguanta**: `cross(p,a,b)` llega a $4C^2$: con `ll` entra hasta $C \approx 1{,}5 \cdot 10^9$; con `long double` el signo es exacto hasta $C \approx 2{,}1 \cdot 10^9$; con `double`, solo hasta $\approx 2 \cdot 10^7$ (tabla) · **Complejidad**: $O(1)$.

`reflect`, `foot`

```
P reflect(P p, Line l);   // espejo de p respecto de la RECTA l
P foot(P p, Line l);      // pie de la perpendicular: el punto de l mas cercano a p
```

`foot` es el **punto medio** entre `p` y `reflect(p, l)` — por eso el código es literalmente `(p + reflect(p,l)) / 2`.

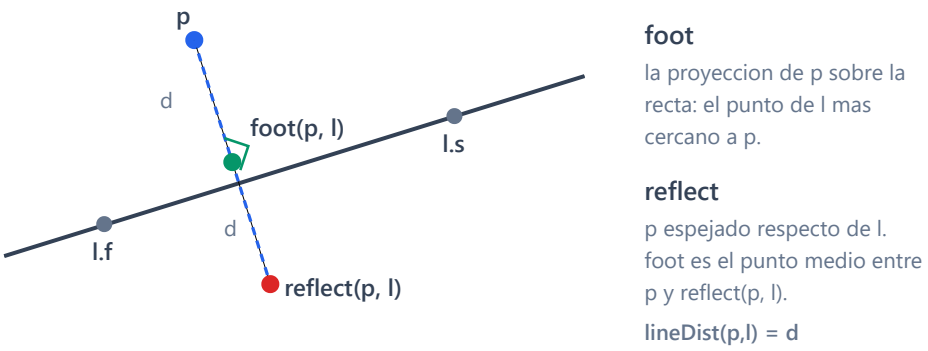


Figura 3. foot y reflect

Restricciones: `l` es una **recta infinita**, no un segmento — `foot` puede caer fuera de `[l.f, l.s]`. `l.f != l.s`: con una recta nula da `(nan, nan)` sin avisar · **Tipo**: `db` · **Aguanta**: el resultado es un punto **calculado**: no lo pases a `onSeg` ni a `sgn` con $C \geq 10^5$ (por qué) · **Complejidad**: $O(1)$.

`onSeg`

```
bool onSeg(P p, Line l);
```

¿`p` está sobre el **segmento** `l`? Usa `sgn`, así que es tolerante a `EPS`.

Restricciones: admite segmento degenerado (`l.f == l.s`), y ahí da `true` solo si `p` es ese punto · **Tipo**: cualquiera · **Aguanta**: con `p` **de entrada**, igual que `cross`; con `p` **calculado** (salida de `lineIsect`, `foot`, `isect`…) **falla desde** $C \approx 10^5$ **y es inservible desde** 10^6 — 80% de falsos negativos (tabla) · **Complejidad**: $O(1)$.

```
onSeg({1,1}, {{0,0},{2,2}})  ->  true
onSeg({0,0}, {{0,0},{0,0}})  ->  true      // segmento degenerado, p es el punto
onSeg({1,1}, {{0,0},{0,0}})  ->  false
```

`AngleCmp.h`

Requiere: `Point.h`

`half`

```
int half(P x);
```

Parte el plano en dos mitades de exactamente π cada una, que es lo que hace posible ordenar por ángulo sin `atan2`.

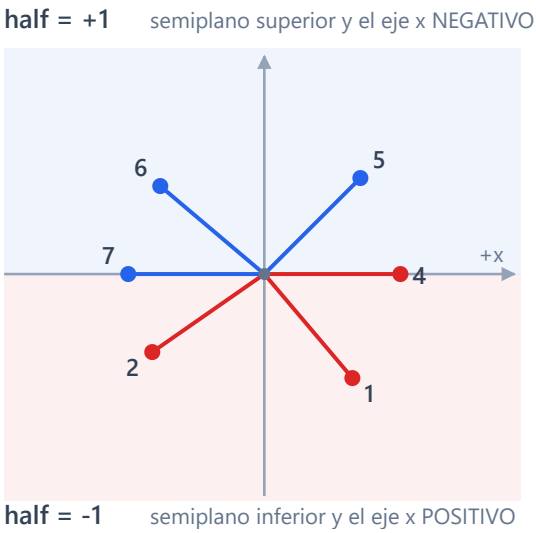


Figura 4. los dos semiplanos de half

valor	qué cae ahí
`-1`	semiplano inferior **y el eje x positivo**
`0`	el origen
`1`	semiplano superior **y el eje x negativo**

Restricciones: ninguna · **Tipo**: cualquiera · **Aguanta**: no multiplica: nunca desborda · **Complejidad**: $O(1)$.

```
half({5,0})   ->  -1      // eje x positivo va ABAJO
half({-5,0})  ->   1      // eje x negativo va ARRIBA
half({0,0})   ->   0
```

> **No lo declares** `bool`. Devuelve tres valores; con `bool` el orden sale mal en silencio. El header lo avisa.

`angleCmp`

```
bool angleCmp(P a, P b);
vP v; sort(all(v), angleCmp);
```

Ordena por ángulo antihorario alrededor del **origen**, con el mismo orden que `atan2`: rango $(-\pi, \pi]$, así que el eje x negativo queda **al final**.

Restricciones: **sacá el origen antes de ordenar** — `half({0,0})` da `0`, que lo mete en un grupo propio entre los otros dos y el resultado no tiene sentido · **Tipo**: `ll` o `db`, los dos; solo usa `sgn` y `cross` · **Aguanta**: usa `cross(a,b)` de dos vectores; si son diferencias de puntos con $|c| \le C$, igual que `cross(p,a,b)` · **Complejidad**: $O(1)$ por comparación, $O(n \log n)$ el sort.

```
// 8 direcciones, desordenadas
{(-1,-1) (0,1) (1,-1) (-1,0) (1,0) (0,-1) (-1,1) (1,1)}
  sort(angleCmp) ->
{(-1,-1) (0,-1) (1,-1) (1,0) (1,1) (0,1) (-1,1) (-1,0)}
   -135    -90    -45     0    45    90    135    180      <- atan2, creciente

// con el origen adentro, el resultado NO sirve:
{(1,0) (0,0) (-1,0) (0,-1)}  sort(angleCmp) ->  {(0,-1) (1,0) (0,0) (-1,0)}
                                                  ^^^^^ en el medio
```

> **No uses `atan2` directo.** Es más lento y pierde precisión; el header trae dos submissions de Codeforces como evidencia.

`SegDist.h`

Requiere: `Point.h`

`lineDist`

```
T lineDist(P p, Line l);
```

Distancia de `p` a la **recta infinita** por `l.f` y `l.s`.

Restricciones: `l.f != l.s` — con recta nula devuelve `nan` sin avisar · **Tipo**: `db` · **Aguanta**: error relativo $\sim 10^{-18}$ · **Complejidad**: $O(1)$.

`segDist`

```
T segDist(P p, Line l);
```

Distancia de `p` al **segmento** `l`. Considera tres casos: la proyección cae antes del extremo, después, o adentro.

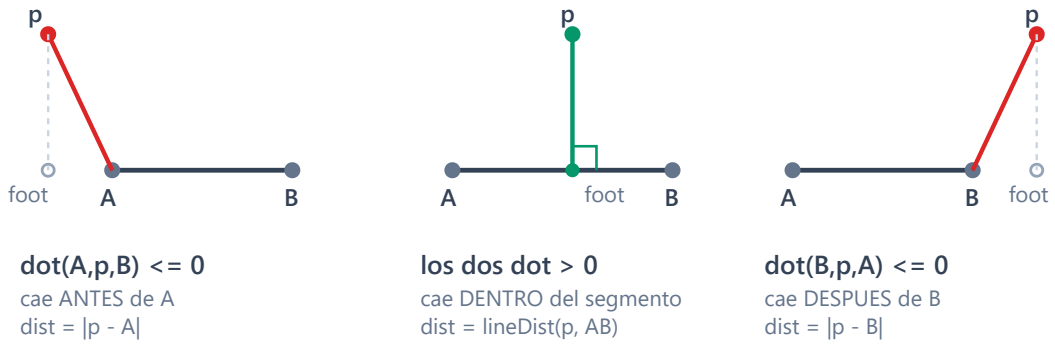


Figura 5. los tres casos de segDist

Restricciones: **admite segmento degenerado** — ahí devuelve la distancia al punto, no `nan` · **Tipo**: `db` · **Aguanta**: error relativo $\sim 10^{-18}$ · **Complejidad**: $O(1)$.

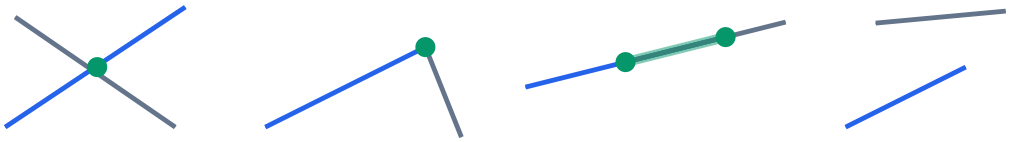
> **El error clásico: usar `lineDist` donde va `segDist`.** Da un valor **menor** y nada avisa.

```
lineDist({5,0}, {{0,0},{1,0}})  ->  0     // la recta y=0 pasa por (5,0)
segDist ({5,0}, {{0,0},{1,0}})  ->  4     // el segmento termina en (1,0)

lineDist({0,1}, {{0,0},{0,0}})  ->  nan   // recta nula
segDist ({0,1}, {{0,0},{0,0}})  ->  1     // segmento nulo: distancia al punto
```

`SegIsect.h`

Requiere: `Point.h`



1 punto
cruce propio
strictIsect lo encuentra

1 punto
se tocan en un extremo
strictIsect da VACIO

2 puntos
colineales, se SOLAPAN
devuelve los dos extremos

vacio
no se tocan

Figura 6. los casos de intersección de segmentos

`lineIsect`

```
vP lineIsect(Line a, Line b);
```

Corte de dos **rectas infinitas**.

devuelve	cuándo
`{punto}`	se cortan en uno solo
`{b.f, b.s}`	son **la misma recta** (infinitos puntos)
`{}`	paralelas distintas

Restricciones: `a.f != a.s` y `b.f != b.s` · **Tipo**: `db` (divide) · **Aguanta**: la decisión (se cortan / paralelas / misma recta) es exacta con entrada entera hasta el umbral de `cross`; el punto devuelto es **calculado** — no lo pases a `onSeg` con $C \geq 10^5$ · **Complejidad**: $O(1)$.

```
lineIsect({{0,0},{2,2}}, {{0,2},{2,0}})  ->  {(1,1)}
lineIsect({{0,0},{1,0}}, {{0,1},{1,1}})  ->  {}                 // paralelas
lineIsect({{0,0},{1,0}}, {{2,0},{3,0}})  ->  {(2,0), (3,0)}     // misma recta
```

> La comparación interna va con `sgn`, no con `==`: con `T = db` dos cross nunca son exactamente iguales.

`strictIsect`

```
vP strictIsect(Line a, Line b);
```

El punto donde los dos **segmentos se cruzan de verdad**, en el **interior** de ambos. Tocarse en una punta **no** cuenta.

Restricciones: igual que `lineIsect` · **Tipo**: `db` (el punto sale de una división) · **Aguanta**: igual que `lineIsect` · **Complejidad**: $O(1)$.

```
strictIsect({{0,0},{2,2}}, {{0,2},{2,0}})  ->  {(1,1)}
strictIsect({{0,0},{1,0}}, {{1,0},{2,0}})  ->  {}       // punta con punta: NO cuenta
```

`segIsect`

```
vP segIsect(Line a, Line b);
```

Intersección de dos **segmentos**, incluyendo puntas y solapamientos. **Admite segmentos degenerados.**

devuelve	cuándo
`{}`	no se tocan
`{punto}`	se cruzan, o se tocan en una punta
`{p, q}`	se **solapan** en un tramo (sus dos extremos)

Restricciones: ninguna; `a` y `b` pueden ser degenerados · **Tipo**: `db` · **Aguanta**: igual que `lineIsect`; sus `onSeg` internos son sobre extremos **de entrada**, así que no sufren el problema del `EPS` · **Complejidad**: $O(1)$.

```
segIsect({{0,0},{1,0}}, {{2,0},{3,0}})  ->  {}               // disjuntos
segIsect({{0,0},{1,0}}, {{1,0},{2,0}})  ->  {(1,0)}          // punta con punta
segIsect({{0,0},{3,0}}, {{1,0},{2,0}})  ->  {(1,0), (2,0)}   // solapados
```

Casos degenerados

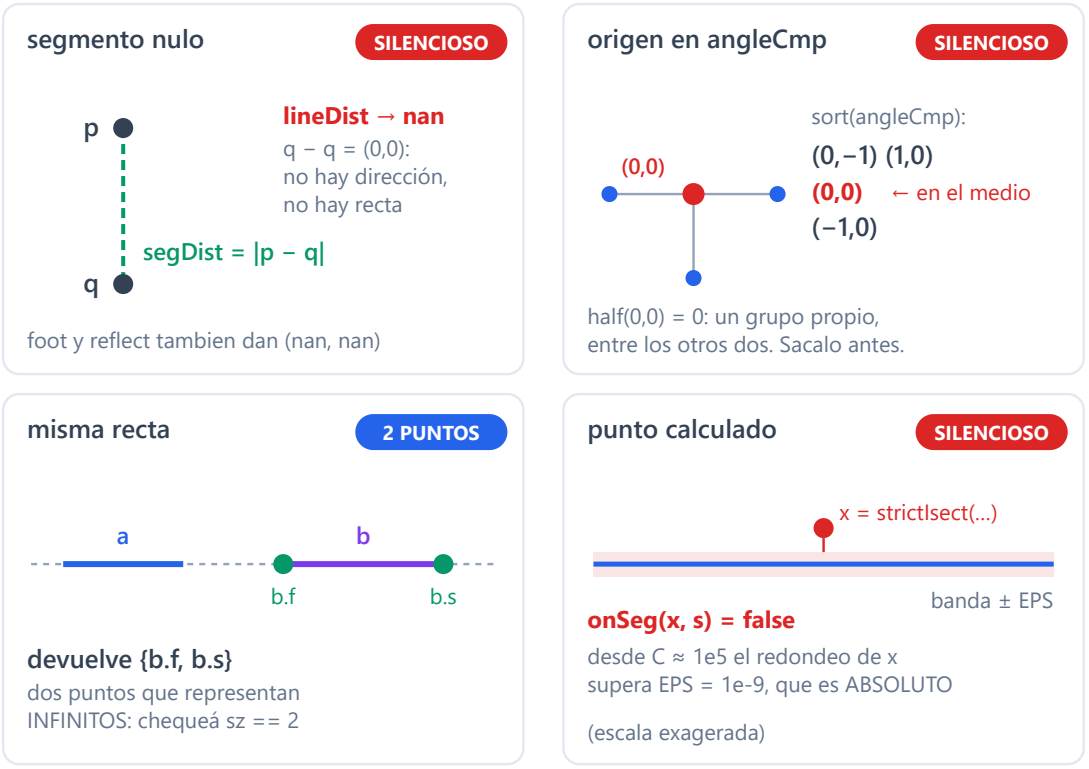


Figura 7. casos degenerados de Primitives

Rojo = falla **en silencio**; naranja = `assert`; azul = resultado válido que rompe una suposición común.

Todas las salidas son de correr el código.

llamada	resultado	
`abs({0,0})`	`0`	ok
`arg({0,0})`	`0`	ok, pero el origen no tiene ángulo
`unit({0,0})`	`(nan, nan)`	silencioso
`foot(p, {q,q})`	`(nan, nan)`	silencioso
`lineDist(p, {q,q})`	`nan`	silencioso
`segDist(p, {q,q})`	la distancia a `q`	**no** es nan
`onSeg(q, {q,q})`	`true`	ok
`onSeg(p, {q,q})` con `p != q`	`false`	ok
`sort(angleCmp)` con el origen adentro	**orden sin sentido**	silencioso
`lineIsect` de la misma recta	`{b.f, b.s}`	2 puntos, no 1
`segIsect` de solapados	`{p, q}`	2 puntos, no 1
`onSeg(x, s)` con `x` **calculado** y $C \geq 10^5$	`false` aunque esté encima	silencioso — 80% con $C = 10^6$

> **Ningún método de esta carpeta tiene** `assert`. Todo lo que sale mal sale como `nan` o como un valor plausible pero incorrecto. Si tu entrada puede traer puntos repetidos, chequealo vos.

Ejemplo completo

Clasificar un punto contra un segmento, con las salidas intermedias:

```
P a = {0,0}, b = {4,0}, p = {6,3};

cout << sgn(cross(a,b,p)) << "\n";   //  1      p esta a la IZQUIERDA de a->b
cout << cross(a,b,p) << "\n";        //  12     = 2 * area del triangulo abp
cout << dot(b,a,p) << "\n";          // -8      el angulo EN b
cout << sgn(dot(b,a,p)) << "\n";     // -1      OBTUSO en b
                                     //         => la proyeccion cae PASADO b

cout << lineDist(p, {a,b}) << "\n";  //  3      la recta y = 0
cout << segDist (p, {a,b}) << "\n";  //  3.6056 = |p - b|, porque cae pasado b

P f = foot(p, {a,b});
cout << f << "\n";                   //  (6,0)  <- FUERA del segmento [a,b]
cout << onSeg(f, {a,b}) << "\n";     //  0      por eso segDist != lineDist

P r = reflect(p, {a,b});
cout << r << "\n";                   //  (6,-3)
cout << (p + r) / 2 << "\n";         //  (6,0)  el punto medio ES foot
```

Los mismos degenerados, en la misma rutina:

```
Line nulo = {{2,2},{2,2}};
cout << lineDist(p, nulo) << "\n";   //  nan       <- sin aviso
cout << segDist (p, nulo) << "\n";   //  4.1231    = |p - (2,2)|, correcto
cout << foot(p, nulo) << "\n";       //  (nan,nan) <- sin aviso
```

Verificación

rutina	casos	contra
`segIsect`, `strictIsect`, `onSeg`	140.000	`cross` exacto en `ll`, C de 2 a 10^9
`segDist`, `lineDist`	60.000	fórmula independiente, error $\leq 4{,}3 \cdot 10^{-16}$
`sgn(cross(...))`	38.000	`__int128`, casos casi degenerados
`angleCmp`	—	orden monótono por `atan2` verificado
umbral de `long double` y `double`	20.000 por celda + búsqueda binaria	signo contra `__int128`

Todos compilan con las flags del `Makefile` (`-Wall -Wextra -Wconversion -Wshadow -Wfloat-equal -D_GLIBCXX_DEBUG`) sin warnings.

Polygons

Convex hull, área, punto en polígono, tangentes, Minkowski, semiplanos.

Manual de consulta: **qué hace cada método, qué le podés pasar y hasta dónde aguanta.** Los problemas donde practicar están en PROBLEMS.md.

Índice

- Cómo se arma la submission
- Dos convenciones
- `ConvexHull.h`
  - `ulHull`
  - `hullInd`
  - `hull`
- `ConvexHullGraham.h`
- `PolygonCenArea.h`
  - `cenArea`
- `InPolygon.h`
  - `inPoly`
- `HullDiameter.h`
  - `diameter2`
- `HullTangents.h`
  - `lower`, `better`
  - `tangent`
- `MinkowskiSum.h`
  - `minkowski_sum`
- `LineHull.h`
  - `extrVertex`
  - `lineHull`
- `HalfPlaneIsect.h`
  - `Ray`
  - `halfPlaneIsect`
- Casos degenerados
- Ejemplo completo
- Verificación
- Lo que no está

Cómo se arma la submission

Copiás `templateHeavy.cpp` en `E.cpp` y pegás el **código** del header en la sección de template.

> **Los `#include` no van en la solución.** El campo `Requiere:` es solo un indicador de qué más pegar, y en qué orden.

si pegás	pegá antes
`ConvexHull.h` *(o `ConvexHullGraham.h`)*	`Point.h`
`PolygonCenArea.h`, `InPolygon.h`, `HullTangents.h`, `MinkowskiSum.h`, `LineHull.h`	`Point.h`
`HalfPlaneIsect.h`	`Point.h`, **`AngleCmp.h`**
`HullDiameter.h`	`Point.h`, **un hull** (llama a `hull()`)

> **`ConvexHull.h` y `ConvexHullGraham.h` son alternativas, no complementos.** Los dos definen `hullInd` y `hull`; pegar ambos da `redefinition of 'vi hullInd(const vP&)'`. **Usá `ConvexHull.h`.**

Dos convenciones

1. El hull es ESTRICTO. Descarta los puntos colineales del borde: de tres puntos alineados sobre una arista, solo quedan los dos extremos.

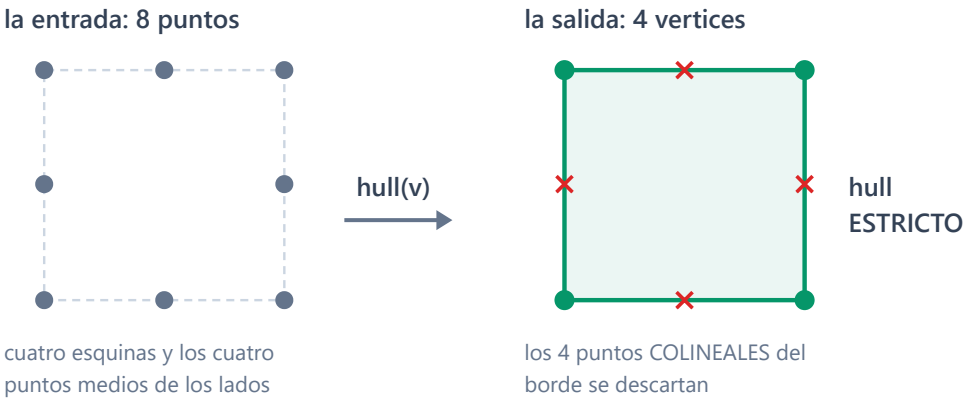


Figura 8. hull estricto

2. `hull` es exacto, `inPoly` usa `EPS`. El `cross` del hull va crudo, sin `sgn` — está pensado para coordenadas enteras. `inPoly` detecta el borde con `onSeg`, que sí pasa por `sgn`.

`ConvexHull.h`

Requiere: `Point.h`

`ulHull`

```
pair<vi,vi> ulHull(const vP& v);
```

Las dos cadenas del hull como **índices**: `.f` la de **arriba**, `.s` la de **abajo**. Las dos van de izquierda a derecha y comparten los puntos extremos.

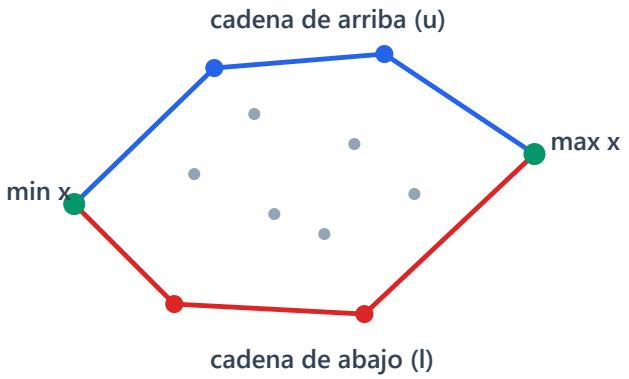


Figura 9. las dos cadenas del hull

Restricciones: ninguna; admite repetidos y colineales · **Tipo**: `ll` recomendado (el `cross` es exacto, sin `EPS`) · **Aguanta**: igual que `cross(p,a,b)`: con `ll` hasta $C \approx 1{,}5 \cdot 10^9$, con `long double` hasta $\approx 2{,}1 \cdot 10^9$ · **Complejidad**: $O(n \log n)$, domina el sort.

Sirve suelta para el **Convex Hull Trick** geométrico, o para recorrer solo "el techo" de una nube.

`hullInd`

```
vi hullInd(const vP& v);
```

Los **índices** de los vértices del hull, en orden **antihorario (CCW)**.

Restricciones: ninguna · **Tipo**: igual que `ulHull` · **Aguanta**: igual que `ulHull` · **Complejidad**: $O(n \log n)$.

```
hullInd({{0,0},{1,1},{2,2},{3,3}})  ->  {0,3}     // colineales: los 2 extremos
```

Usala cuando necesites saber **qué puntos originales** quedaron, no solo sus coordenadas.

`hull`

```
vP hull(const vP& v);
```

Los **vértices** del hull en orden **antihorario (CCW)**.

Restricciones: ninguna · **Tipo**: igual que `ulHull` · **Aguanta**: igual que `ulHull` · **Complejidad**: $O(n \log n)$.

```
hull({{0,0}})                    ->  {(0,0)}                 // 1 punto
hull({{0,0},{1,1}})              ->  {(0,0), (1,1)}          // 2 puntos
hull({{2,2},{2,2},{2,2}})        ->  {(2,2)}                 // todos iguales
hull({{0,0},{1,1},{2,2},{3,3}})  ->  {(0,0), (3,3)}          // colineales
hull(100 puntos colineales)      ->  {(0,0), (99,0)}         // 2, no 100
hull({{0,0},{2,0},{4,0},{0,4}})  ->  {(0,0), (4,0), (0,4)}   // ESTRICTO: (2,0) afuera
```

> **El hull puede salir con menos de 3 vértices.** Si tu código asume un polígono, chequeá `sz(h) >= 3` antes.

`ConvexHullGraham.h`

```
vi hullInd(const vP& v);
vP hull(const vP& v);
```

Lo mismo por **Graham scan**: ordena por ángulo alrededor del punto más bajo en vez de por coordenada. Misma salida, misma complejidad, verificado contra la misma referencia.

	`ConvexHull.h`	`ConvexHullGraham.h`
algoritmo	cadena monótona (Andrew)	Graham scan
extra	expone `ulHull`	—
líneas	34	27

Usá `ConvexHull.h` salvo que tengas una razón concreta.

`PolygonCenArea.h`

Requiere: `Point.h`

`cenArea`

```
pair<P,T> cenArea(const vP& v);
```

`{centroide, área con signo}` de un polígono **simple**. El signo es **positivo si viene en CCW**, negativo si en CW.



Figura 10. el signo del área

Para el área a secas: `abs(cenArea(v).s)`.

Restricciones: $n \geq 3$ (`assert`); el polígono tiene que ser **simple** — sin aristas que se crucen. Con uno no simple devuelve basura sin avisar · **Tipo**: con `T = ll` **las dos divisiones truncan**, la del centroide y la del área (el acumulador interno vale $2A$ y se devuelve `/2`): con enteros acumulá el doble del área a mano · **Aguanta**: con `ll`, el área es exacta hasta $C = 10^9$, pero **el centroide desborda en silencio desde** $C \approx 1{,}4 \cdot 10^6$ (medido: bien a $1{,}4 \cdot 10^6$, basura a $2 \cdot 10^6$; su acumulador crece como C^3). Con `long double` no hay desborde · **Complejidad**: $O(n)$.

```
cenArea({{0,0},{4,0},{4,4},{0,4}})   ->  centro (2,2), area =  16   // CCW
cenArea(el mismo invertido)          ->                area = -16   // CW

// polígono NO simple (un moño) — precondición rota:
cenArea({{0,0},{4,0},{0,4},{4,4}})   ->  area = 0, centro (nan,-inf)
```

> El signo es una forma barata de **detectar la orientación**: `sgn(cenArea(v).s) > 0` ⟺ CCW.

`InPolygon.h`

Requiere: `Point.h`

`inPoly`

```
int inPoly(const P& p, const vP& poly);
```

Dónde cae `p`: **`-1` adentro, `0` sobre el borde, `1` afuera**. Por *ray casting*: cuenta cuántas veces un rayo cruza el borde.

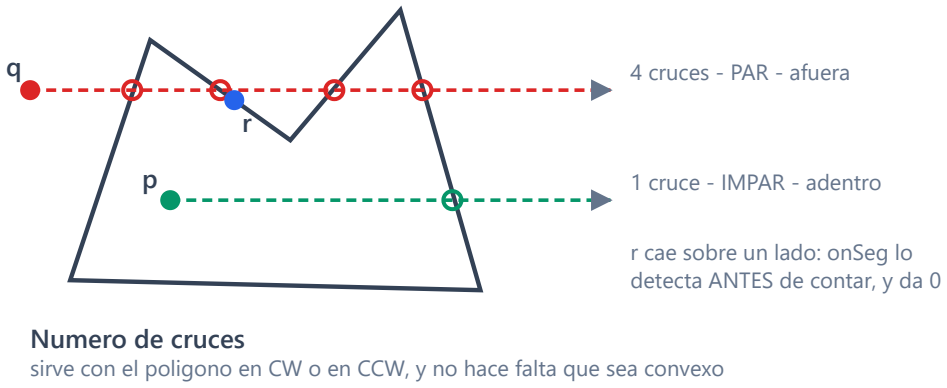


Figura 11. ray casting

Restricciones: el polígono tiene que ser **simple**, pero **no** hace falta que sea convexo ni que venga en un orden particular — **anda igual en CW y en CCW** · **Tipo**: `db` o `ll` · **Aguanta**: con `p` de entrada, igual que `cross`; si `p` es un punto **calculado**, el borde (`0`) se pierde desde $C \approx 10^5$ · **Complejidad**: $O(n)$ por consulta.

```
// cuadrado (0,0)(4,0)(4,4)(0,4)
inPoly({2,2}, sq)  ->  -1    // adentro
inPoly({2,0}, sq)  ->   0    // sobre una arista
inPoly({0,0}, sq)  ->   0    // sobre un vertice
inPoly({9,9}, sq)  ->   1    // afuera
inPoly({2,2}, sq_invertido)  ->  -1    // el MISMO resultado en CW

// polígono en L (no convexo): (0,0)(4,0)(4,2)(2,2)(2,4)(0,4)
inPoly({1,1}, L)   ->  -1    // adentro
inPoly({3,3}, L)   ->   1    // en la muesca: afuera
inPoly({2,2}, L)   ->   0    // el vertice reflex
```

> Para un **convexo** con muchas consultas, $O(n)$ por consulta es caro. La versión $O(\log n)$ sale de `tangent` o de una binaria angular; no está hecha acá (Benq tampoco la tiene).

`HullDiameter.h`

Requiere: `Point.h`, **un hull**

`diameter2`

```
T diameter2(vP ps);
```

El **cuadrado** de la mayor distancia entre dos puntos del conjunto, por *rotating calipers*.

> **No le pases el hull: pasale los puntos crudos.** Lo calcula internamente. **Devuelve el cuadrado**, no la distancia: `sqrt(diameter2(ps))`.

Restricciones: $n \geq 1$ · **Tipo**: `ll` conviene — el cuadrado queda exacto · **Aguanta**: el resultado llega a $8C^2$; con `ll` entra hasta $C \approx 1{,}07 \cdot 10^9$ (con $C = 10^9$: $8 \cdot 10^{18} < 9{,}2 \cdot 10^{18}$) · **Complejidad**: $O(n \log n)$ en total, $O(n)$ dado el hull.

```
diameter2({{0,0}})              ->  0
diameter2({{0,0},{3,4}})        ->  25        // 5^2
diameter2({{1,1},{1,1},{1,1}})  ->  0
diameter2(100 colineales 0..99) ->  9801      // 99^2
```

`HullTangents.h`

Requiere: `Point.h`

`lower`, `better`

```
bool lower;                    // variable GLOBAL
bool better(P a, P b, P c);
```

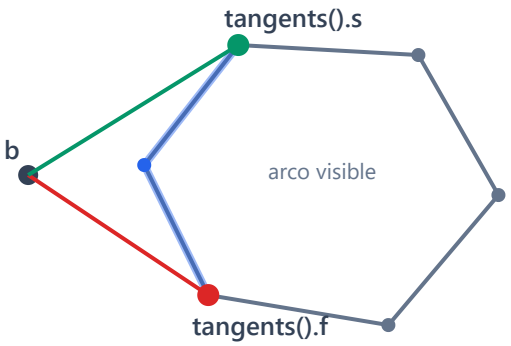
`lower` elige **cuál de las dos tangentes** busca `tangent`. Son internas, pero `lower` hay que setearla antes de cada llamada.

> **Es una variable global.** Si la olvidás, `tangent` devuelve la tangente que quedó de la llamada anterior.

`tangent`

```
int tangent(const vP& a, P b);
```

El **índice** del vértice donde la tangente desde `b` toca al convexo `a`.



tangents(a, b)

Desde un punto b ESTRICTAMENTE afuera, devuelve los indices de los dos vertices de tangencia.

Todo el convexo queda de un solo lado de cada tangente: eso es lo que las define.

Entre las dos queda el arco que se VE desde b, en O(log n).

Figura 12. tangentes y arco visible

Restricciones: `a` tiene que ser **convexo, en CCW y sin tres vértices colineales** — el hull estricto de `ConvexHull.h` cumple las tres. `b` **tiene que estar estrictamente afuera**: con `b` adentro devuelve un índice **plausible pero incorrecto**, sin avisar · **Tipo**: `db` o `ll` · **Aguanta**: igual que `cross(p,a,b)` · **Complejidad**: $O(\log n)$.

```
// cuadrado CCW (0,0)(4,0)(4,4)(0,4), punto (10,2) afuera
lower = 1;  tangent(sq, {10,2})  ->  2
lower = 0;  tangent(sq, {10,2})  ->  1

lower = 1;  tangent(sq, {2,2})   ->  3    // punto ADENTRO: basura silenciosa
```

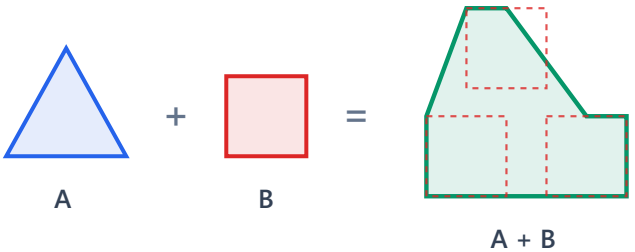
`MinkowskiSum.h`

Requiere: `Point.h`

`minkowski_sum`

```
vP minkowski_sum(vP a, vP b);
```

La **suma de Minkowski** $\{p+q : p \in A,\ q \in B\}$ de dos convexos. El resultado es convexo y queda en CCW.



minkowski_sum

El conjunto de TODAS las sumas p + q con p en A y q en B.

Equivale a barrer B con su origen recorriendo todo A: por eso el resultado es A "engordado".

Dos convexos A y B se tocan si y solo si el ORIGEN cae dentro de A + (-B)

Figura 13. suma de Minkowski

Restricciones: `a` y `b` tienen que ser **convexos y venir en CCW** — no se verifica; con otro orden el resultado es basura · **Tipo**: `db` o `ll` · **Aguanta**: las coordenadas se **suman**, así que el resultado llega a $2C$ · **Complejidad**: $O(n+m)$ — lineal, porque fusiona las aristas ya ordenadas por ángulo.

```
minkowski_sum(sq, {{0,0},{2,0},{0,2}})  ->  {(0,0),(6,0),(6,4),(4,6),(0,6)}   // 5 vertices
minkowski_sum(sq, {{10,10}})            ->  {(10,10),(14,10),(14,14),(10,14)} // traslacion
minkowski_sum(sq, {})                   ->  {}
```

> **El uso clásico:** dos convexos A y B chocan $\Leftrightarrow$ el origen está dentro de $A \oplus (-B)$.

`LineHull.h`

Requiere: `Point.h`

`extrVertex`

```
int extrVertex(const vP& poly, P nrm);
```

El índice del vértice de **máxima proyección** sobre la dirección `nrm`.

> El parámetro se llama `nrm` y no `dir` porque `dir()` ya es una función de `Point.h`.

Restricciones: `poly` **convexo en CCW** · **Tipo**: cualquiera · **Aguanta**: cada término es `nrm` por una diferencia de vértices: con `ll`, hasta $4C \cdot \max|nrm|$ · **Complejidad**: $O(\log n)$.

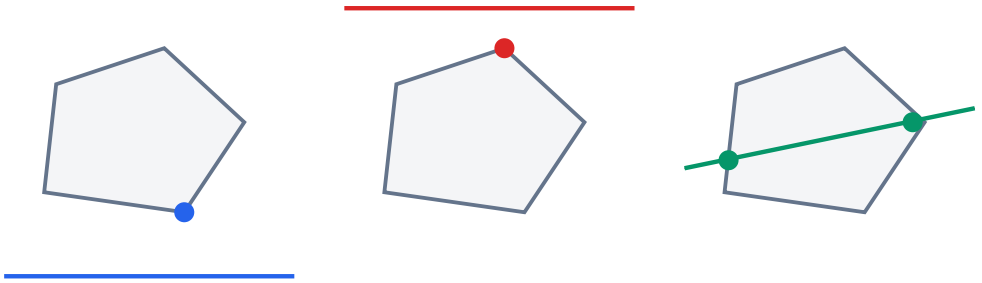
```
// cuadrado CCW (0,0)(4,0)(4,4)(0,4)
extrVertex(sq, {1,0})   ->  1     // (4,0)
extrVertex(sq, {0,1})   ->  2     // (4,4)
extrVertex(sq, {-1,0})  ->  3     // (0,4)
```

Es un **programa lineal de dos variables** resuelto en $O(\log n)$: maximizar $\mathbf{nrm} \cdot p$ sobre el convexo.

`lineHull`

```
pair<int,vi> lineHull(Line line, const vP& poly);
```

Cómo corta una **recta infinita** a un convexo.



.f = 1
todos de un lado

.f = -1
todos del otro

.f = 0
la recta ATRAVIESA

en los dos primeros .s trae los vertices MAS CERCANOS a la recta; en el tercero, el rango de vertices que quedan de un lado

Figura 14. los tres casos de lineHull

`.f`	significa	`.s`
`1`	todo el polígono está **de un lado**	los vértices más cercanos a la recta
`-1`	todo del **otro lado**	ídem
`0`	la recta **lo atraviesa**	el rango de vértices que quedan de un lado

Restricciones: `poly` **convexo en CCW**, $n > 1$ (`assert`) · **Tipo**: cualquiera · **Aguanta**: igual que `cross(p,a,b)` , si la recta también cumple $|c| \leq C$ · **Complejidad**: $O(\log n)$.

```
// cuadrado CCW (0,0)(4,0)(4,4)(0,4)
lineHull({{-5,2},{9,2}}, sq)  ->  { 0, {0,1} }   // y=2 lo atraviesa
lineHull({{-5,9},{9,9}}, sq)  ->  {-1, {2,3} }   // y=9 no lo toca
lineHull({{-5,4},{9,4}}, sq)  ->  {-1, {2,3} }   // y=4 TANGENTE por arriba
lineHull({{-5,0},{9,0}}, sq)  ->  { 1, {0,1} }   // y=0 TANGENTE por abajo
```

> **Tangente y "no toca" devuelven el mismo signo.** Para distinguirlas hay que mirar si los vértices de `.s` están realmente sobre la recta.

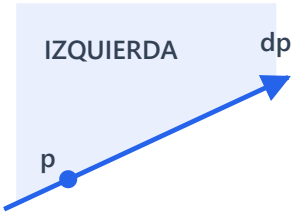
`HalfPlaneIsect.h`

Requiere: `Point.h` , **`AngleCmp.h`**

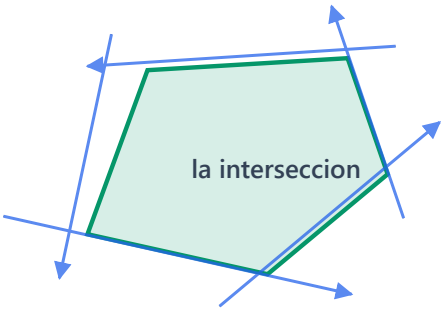
`Ray`

```
struct Ray { P p, dp; };    // origen, direccion
```

Un semiplano es **el área a la IZQUIERDA** del rayo `{p, dp}` .



Un semiplano se da como un rayo {p, dp}: el area valida es la que queda a su IZQUIERDA.



halfPlaneIsect devuelve los vertices de esa region, en CCW

Figura 15. semiplanos

`halfPlaneIsect`

```
vP halfPlaneIsect(V<Ray> rays, bool add_bounds = false);
```

Los vértices de la intersección de los semiplanos, en CCW.

Restricciones: la intersección tiene que ser **acotada y de área no nula**. Con menos de 3 rayos devuelve `{}`. **Si la región es no acotada el resultado trae `inf` y `nan`, sin avisar** — para eso está `add_bounds` · **Tipo**: `db` · **Aguanta**: medido contra Sutherland–Hodgman, el error **relativo** del área es $\leq 5 \cdot 10^{-15}$ a cualquier escala hasta 10^9; el error **absoluto** crece como área × 10^{-15}, por eso el header dice "precisa hasta 10^5" para áreas enteras. Rayos casi paralelos no están medidos · **Complejidad**: $O(n \log n)$, domina el sort angular.

`add_bounds = true` agrega 4 rayos que acotan todo al rectángulo $[0, 10^9] \times [0, 10^9]$.

```
// triangulo (0,0) (6,0) (0,6)
V<Ray> r = {{{0,0},{1,0}}, {{6,0},{-1,1}}, {{0,6},{0,-1}}};
halfPlaneIsect(r)  ->  {(0,0), (6,0), (0,6)}      cenArea(...).s = 18

halfPlaneIsect({dos rayos})     ->  {}                       // menos de 3
halfPlaneIsect({franja abierta})->  {(0,0),(0,1),(inf,nan)}  // NO ACOTADA
halfPlaneIsect({dos rayos}, true) ->
      {(0,0), (1e9,0), (1e9,1), (0,1)}                       // acotado por el rectangulo
```

> **Si no estás seguro de que la región sea acotada, pasá `add_bounds = true`**. Es la diferencia entre un polígono y un `nan`.

Casos degenerados

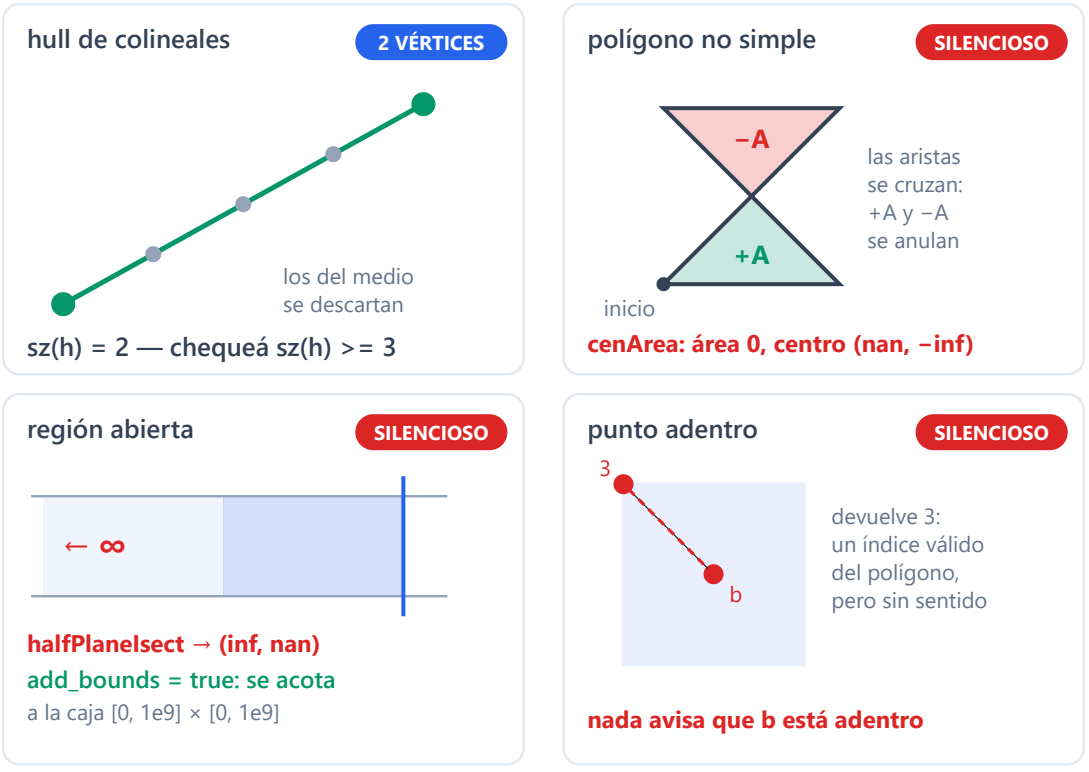


Figura 16. casos degenerados de Polygons

Rojo = falla **en silencio**; naranja = `assert`; azul = resultado válido que rompe una suposición común.

Todas las salidas son de correr el código.

situación	qué pasa	
`hull` de 1 punto	`{p}`	ok, pero no es polígono
`hull` de todos iguales	`{p}`	ok
`hull` de colineales	los **2** extremos	ok
`cenArea` con $n < 3$	**`assert`**	avisa
`cenArea` de un polígono **no simple**	área 0, centro `(nan, -inf)`	**silencioso**
`cenArea` con `T = ll`	el centroide **trunca**	silencioso
`cenArea` con `T = ll` y $C \geq 1{,}4 \cdot 10^6$	el centroide **desborda**: basura	**silencioso** — el área sigue bien
`inPoly` sobre vértice o arista	`0`	ok
`inPoly` en CW	igual que en CCW	ok
`diameter2` de 1 punto	`0`	ok
`tangent` con el punto **adentro**	índice **plausible pero falso**	**silencioso**
`tangent` sin setear `lower`	la tangente de la llamada anterior	**silencioso**
`minkowski_sum` con un vacío	`{}`	ok
`minkowski_sum` con entrada en CW	basura	**silencioso**
`lineHull` con $n \leq 1$	**`assert`**	avisa
`lineHull` tangente vs. no toca	**mismo signo**	hay que distinguirlas a mano
`halfPlaneIsect` con < 3 rayos	`{}`	ok
`halfPlaneIsect` región **no acotada**	`(inf, nan)`	**silencioso** — usá `add_bounds`

> Solo `cenArea` y `lineHull` tienen `assert`. Las precondiciones de orientación (CCW) y de "punto afuera" **no se verifican en ningún lado**.

Ejemplo completo

Del conjunto de puntos crudo a todas las consultas, con las salidas intermedias:

```
vP pts = {{0,0},{4,0},{4,4},{0,4},{2,2},{2,0}};   // (2,2) interior, (2,0) colineal

vP h = hull(pts);
cout << sz(h) << "\n";            // 4     (2,2) y (2,0) descartados
each(p, h) cout << p << " ";      // (0,0) (4,0) (4,4) (0,4)   <- CCW

auto ca = cenArea(h);
cout << ca.s << "\n";             // 16    area POSITIVA => confirma CCW
cout << ca.f << "\n";             // (2,2) el centroide

cout << inPoly({2,2}, h) << "\n"; // -1    adentro
cout << inPoly({2,0}, h) << "\n"; //  0    sobre la arista de abajo
cout << inPoly({9,9}, h) << "\n"; //  1    afuera

cout << diameter2(pts) << "\n";   // 32    la diagonal al cuadrado, 4^2+4^2
cout << sqrt(diameter2(pts));     // 5.6569

// tangentes: hay que setear lower ANTES de cada llamada
lower = 1; int t1 = tangent(h, {10,2});   // 2
lower = 0; int t2 = tangent(h, {10,2});   // 1
```

El caso degenerado que más cuesta ver:

```
vP mono = {{0,0},{4,0},{0,4},{4,4}};   // un MOÑO: las aristas se cruzan
auto r = cenArea(mono);
cout << r.s << "\n";                   // 0          <- parece area nula
cout << r.f << "\n";                   // (nan,-inf) <- aca se nota

// el chequeo barato: un poligono simple con area 0 es imposible salvo degenerado
if (sgn(r.s) == 0) { /* o es degenerado, o NO es simple */ }
```

Verificación

rutina	casos	contra
`tangent`	131.679	propiedad de tangencia + fuerza bruta $O(n^2)$
`inPoly`	170.000 (104.000 de borde)	*crossing number* exacto, CW y CCW
`cenArea`, `diameter2`, `minkowski_sum`	44.000	*shoelace*, $O(n^2)$ y hull de todas las sumas
`hull`, `hullInd`, `ulHull`	14.800 + 12 bordes	cadena monótona exacta en `ll`, C de 2 a 10^9
`hull` (Graham)	8.000	misma referencia
`extrVertex`, `lineHull`	25.972	fuerza bruta: máximo real, signo, aristas cruzadas
`halfPlaneIsect`	9.500	Sutherland-Hodgman, recorte sucesivo

Los nueve compilan juntos (menos Graham, que es alternativa) con las flags del `Makefile` sin warnings.

Lo que no está

- **Punto en polígono convexo en $O(\log n)$** como rutina propia. `inPoly` es $O(n)$; sale de `tangent` o de una binaria angular en ~10 líneas. Benq tampoco la tiene.
- **`HalfPlaneSet.h`** de cp-notebook (semiplanos *online*, con inserciones). Redefine `T`, `INF` y `Line` (32 errores contra esta librería) y su `Maximize` está marcado UNTESTED. Para el caso offline está `HalfPlaneIsect.h`.

Circles

Intersección, tangentes, circuncentro y círculo mínimo.

Manual de consulta: **qué hace cada método, qué le podés pasar y hasta dónde aguanta.** Los problemas donde practicar están en PROBLEMS.md.

Índice

- Cómo se arma la submission
- Contrato de tipos
- `Circle.h`
  - `in`
  - `arcLength`
- `CircleIsect.h`
  - `isect(Circ, Circ)`
  - `isect(Circ, Line)`
  - `isect_area`
- `CircleTangents.h`
  - `tangent`
  - `external`
  - `internal`
- `Circumcenter.h`
  - `ccCenter`
- `MinEnclosingCirc.h`
  - `mec`
- Casos degenerados
- Ejemplo completo
- Verificación

Cómo se arma la submission

Copiás `templateHeavy.cpp` en `E.cpp` y pegás el **código** del header en la sección de template.

> **Los `#include` no van en la solución.** El campo `Requiere:` es solo un indicador de qué más pegar, y en qué orden.

si pegás	pegá antes
`Circle.h`	`Point.h`
`CircleIsect.h`	`Point.h` , `Circle.h`
`CircleTangents.h`	`Point.h` , `Circle.h`
`Circumcenter.h`	`Point.h` , `Circle.h`
`MinEnclosingCirc.h`	`Point.h` , `Circle.h` , **`Circumcenter.h`**

Contrato de tipos

```
using Circ = pair<P,T>;   // {centro, radio}
```

> **Toda esta carpeta exige `T = db`**. Usa `sqrt`, `acos`, `unit` y `dir`; con `T = ll` todo eso trunca a cero y los resultados salen mal **en silencio**, sin warning ni assert.

`Circle.h`

Requiere: `Point.h`

`in`

```
int in(const Circ& x, const P& y);
```

Dónde cae `y` respecto de `x`: **`-1` adentro, `0` en el borde, `1` afuera**.

Restricciones: ninguna; radio 0 es válido · **Tipo**: `db` · **Aguanta**: robusto también con puntos **calculados**: medido 0% de error hasta $C = 10^7$ con puntos salidos de `isect` (compara distancias, que escalan con C, no con C^2) · **Complejidad**: $O(1)$.

```
in({{0,0},2}, {1,0})  ->  -1    // adentro
in({{0,0},2}, {2,0})  ->   0    // borde (con EPS)
in({{0,0},0}, {0,0})  ->   0    // radio 0: su centro esta "en el borde"
```

`arcLength`

```
T arcLength(const Circ& x, P a, P b);
```

Longitud del arco **más corto** entre `a` y `b` sobre `x`.

> **No es el arco orientado.** El resultado siempre está en $[0, \pi r]$ y **no depende del orden** de `a` y `b`. Si necesitás el arco antihorario (que puede pasar de πr), tenés que calcularlo aparte.

Restricciones: `a` y `b` **deben estar sobre el círculo** — no se verifica · **Tipo**: `db` · **Aguanta**: **los arcos chicos pierden precisión** — el `acos` está mal condicionado cerca de 1: error relativo $2 \cdot 10^{-9}$ a 10^{-6} rad, 1,2% a 10^{-9}, y **devuelve 0** a 10^{-10} · **Complejidad**: $O(1)$.

```
// circulo unidad en el origen
arcLength(c, {1,0},  {0,1})  ->  1.5708   // pi/2
arcLength(c, {0,1},  {1,0})  ->  1.5708   // el MISMO: no mira el sentido
arcLength(c, {1,0},  {1,0})  ->  0.0000
arcLength(c, {1,0}, {-1,0})  ->  3.1416   // pi, el maximo posible
```

El `acos` está *clampeado* a $[-1, 1]$: sin eso, un redondeo que deje el argumento en 1.0000000001 daría `nan`.

`CircleIsect.h`

Requiere: `Point.h`, `Circle.h`

`isect(Circ, Circ)`

```
vP isect(const Circ& x, const Circ& y);
```

Los puntos de corte de dos círculos. **Una tangencia devuelve el mismo punto dos veces**, no uno.

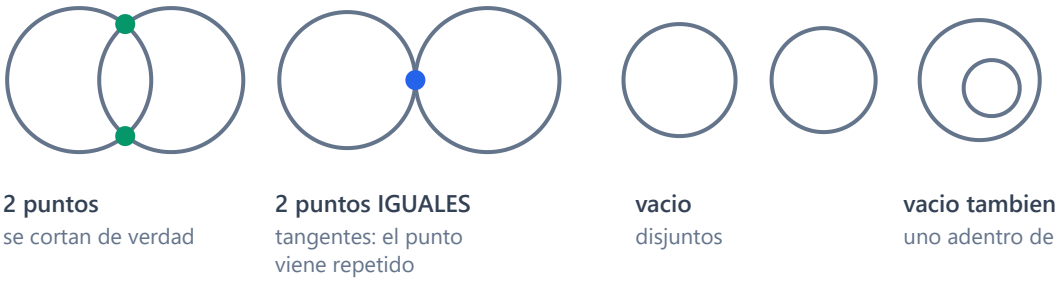


Figura 17. los cuatro casos de intersección de círculos

Restricciones: `x != y` — con dos círculos idénticos hay infinitos puntos y salta un `assert` · **Tipo**: `db` · **Aguanta**: los puntos devueltos son **calculados**: `in()` los reconoce bien hasta 10^7; `onSeg` no · **Complejidad**: $O(1)$.

```
isect({{0,0},2}, {{2,0},2})  ->  {(1,1.7321), (1,-1.7321)}   // secantes: 2 puntos
isect({{0,0},1}, {{2,0},1})  ->  {(1,0), (1,0)}              // tangentes: repetido
isect({{0,0},1}, {{9,0},1})  ->  {}                          // disjuntos
isect({{0,0},1}, {{0,0},2})  ->  {}                          // concentricos
isect({{1,0},0}, {{0,0},1})  ->  {(1,0), (1,0)}              // radio 0 sobre el borde
isect({{9,0},0}, {{0,0},1})  ->  {}                          // radio 0 afuera
```

> El caso de **radio 0** tiene una rama propia. Sin ella se dividía por el radio y salían `nan` en silencio.

`isect(Circ, Line)`

```
vP isect(const Circ& x, const Line& y);
```

Corte de un círculo con una **recta infinita** (no un segmento). Tangente → el mismo punto dos veces.



Figura 18. los tres casos de círculo contra recta

Restricciones: `y.f != y.s` · **Tipo**: `db` · **Aguanta**: igual que `isect(Circ, Circ)` · **Complejidad**: $O(1)$.

```
isect({{0,0},1}, {{-5,0},{5,0}})  ->  {(1,0), (-1,0)}   // secante
isect({{0,0},1}, {{-5,1},{5,1}})  ->  {(0,1), (0,1)}    // tangente: repetido
isect({{0,0},1}, {{-5,9},{5,9}})  ->  {}                 // no toca
```

Para recortar un **segmento**, quedate con los puntos cuyo parámetro sobre el segmento esté en $[0, 1]$. **No uses** `onSeg` sobre estos puntos: son calculados, y con $C \geq 10^5$ `onSeg` da falsos negativos (por qué).

`isect_area`

```
T isect_area(Circ x, Circ y);
```

Área de la intersección de dos discos.

Restricciones: ninguna; cubre contención, tangencia y radio 0 · **Tipo**: `db` · **Aguanta**: sin `nan`: los `acos` están acotados a $[-1, 1]$ y la raíz de Herón a ≥ 0 · **Complejidad**: $O(1)$.

```
isect_area({{0,0},1}, {{0,0},1})  ->  3.1416   // identicos: pi
isect_area({{0,0},3}, {{0,0},1})  ->  3.1416   // uno dentro: area del chico
isect_area({{0,0},1}, {{2,0},1})  ->  0.0000   // tangentes
isect_area({{0,0},1}, {{9,0},1})  ->  0.0000   // disjuntos
isect_area({{0,0},1}, {{1,0},1})  ->  1.2284   // secantes
```

A diferencia de `isect(Circ,Circ)`, **este sí admite círculos idénticos**.

`CircleTangents.h`

Requiere: `Point.h`, `Circle.h`

`tangent`

```
P tangent(P x, Circ y, int t = 0);
```

El punto donde la tangente desde `x` toca a `y`. `t = 0` y `t = 1` son los dos puntos posibles.

Restricciones: `x` **fuera del círculo o sobre el borde.** Con `x` adentro devuelve `(nan, nan)` **sin avisar** · **Tipo**: `db` · **Aguanta**: el borde tiene una tolerancia **relativa**: un `x` adentro por menos de $\approx 5 \cdot 10^{-10}$ del radio cuenta como borde y devuelve el propio `x`; más adentro, `nan`. Por eso funciona con `x` **calculado** sobre el borde, a cualquier escala · **Complejidad**: $O(1)$.

```
tangent({3,0}, {{0,0},1}, 0)  ->  (0.3333,  0.9428)
tangent({3,0}, {{0,0},1}, 1)  ->  (0.3333, -0.9428)
tangent({0.5,0}, {{0,0},1})   ->  (nan, nan)     // punto ADENTRO
tangent({3,0}, {{0,0},0})     ->  (0, 0)         // radio 0: el centro
tangent(c.f + dir(th)*r, c)   ->  el mismo x     // x calculado SOBRE el borde
```

`external`

```
V<pair<P,P>> external(Circ x, Circ y);
```

Las **dos tangentes externas**: cada par es `{punto sobre x, punto sobre y}`.

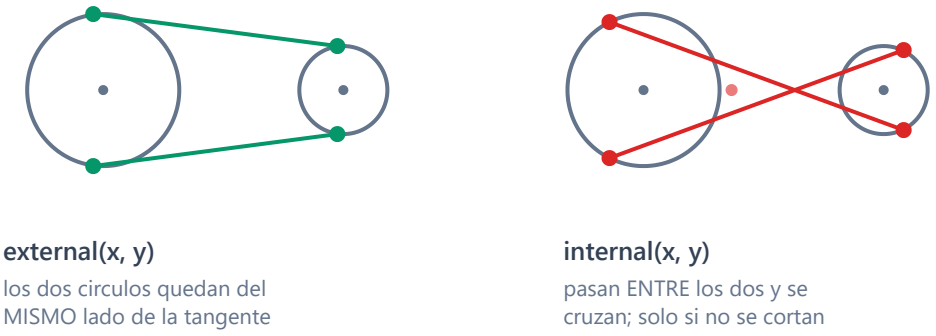


Figura 19. tangentes externas e internas

Restricciones: **ningún círculo puede contener al otro.** Si uno contiene al otro devuelve **dos pares de** `(nan, nan)` — no devuelve vacío, y nada avisa · **Tipo**: `db` · **Aguanta**: radios casi iguales están cubiertos por la rama de `sgn`; círculos tangentes con centros calculados también (el centro de homotecia cae sobre un borde, y `tangent` lo tolera) · **Complejidad**: $O(1)$.

```
external({{0,0},1}, {{4,0},1})  ->  {(0,-1)->(4,-1), (0,1)->(4,1)}
external({{0,0},3}, {{0,0},1})  ->  {(nan,nan)->(nan,nan), (nan,nan)->(nan,nan)}
```

Con **radios iguales** usa una rama aparte (las tangentes son paralelas); si no, usa el centro de homotecia. El `sgn` de esa decisión no es cosmético: con radios casi iguales el centro se va al infinito.

`internal`

```
V<pair<P,P>> internal(Circ x, Circ y);
```

Las **dos tangentes internas**, las que cruzan entre los dos círculos. Es `external` con el radio de `x` negado.

Restricciones: los círculos **no pueden solaparse**; si se solapan, `(nan, nan)` · **Tipo**: `db` · **Aguanta**: igual que `external` · **Complejidad**: $O(1)$.

```
internal({{0,0},1}, {{4,0},1})  ->  {(0.5,0.866)->(3.5,-0.866), (0.5,-0.866)->(3.5,0.866)}
internal({{0,0},1}, {{2,0},1})  ->  {(1,0)->(1,0), (1,0)->(1,0)}   // tangentes internos
```

`Circumcenter.h`

Requiere: `Point.h`, `Circle.h`

`ccCenter`

```
Circ ccCenter(P a, P b, P c);
```

`{circuncentro, circunradio}` del triángulo `abc`: el único círculo que pasa por los tres.

Restricciones: **los tres puntos no pueden ser colineales ni repetirse.** Un `assert` lo corta; sin él el resultado sería `nan` en silencio · **Tipo**: `db` · **Aguanta**: con entrada entera el `assert` salta **solo** con colineales exactos; con puntos casi colineales el radio sale enorme, sin `nan` · **Complejidad**: $O(1)$.

```
ccCenter({0,0}, {4,0}, {0,4})  ->  centro (2, 2),      radio 2.8284
ccCenter({0,0}, {4,0}, {2,3})  ->  centro (2, 0.8333), radio 2.1667
```

> Si tu entrada puede traer ternas degeneradas, filtralas antes:
>
> ```
> if (sgn(cross(a,b,c)) == 0) continue;
> ```

`MinEnclosingCirc.h`

Requiere: `Point.h`, `Circle.h`, `Circumcenter.h`

`mec`

```
Circ mec(vP ps);
```

El **círculo mínimo que contiene todos los puntos** (Welzl).



Figura 20. el MEC queda determinado por 2 o 3 puntos

Restricciones: $n \geq 1$; admite duplicados, todos iguales y todos colineales · **Tipo**: `db` · **Aguanta**: tolerancia **relativa** 10^{-8} (`holgura`): un punto afuera por menos de $r \cdot 10^{-8}$ cuenta como adentro, así que el radio puede salir hasta 10^{-8} relativo más chico. No depende de C · **Complejidad**: $O(n)$ **esperado — es un algoritmo aleatorizado.** El peor caso es $O(n^3)$, pero requiere un orden adversario; `mec` mezcla la entrada para evitarlo.

Toma `ps` **por copia**: la mezcla.

```
mec({{3,3}})                    ->  centro (3,3),     radio 0
mec({{1,1},{1,1},{1,1}})        ->  radio 0
mec({{0,0},{1,1},{2,2},{3,3}})  ->  centro (1.5,1.5), radio 2.1213   // colineales
```

Qué tan esperado es el $O(n)$. Con 200 puntos al azar, contando cuántas veces entra cada bucle:

```
bucle i (reinicia el circulo) :  10 veces
bucle j (fija el 2do punto)   :  43 veces
bucle k (fija el 3ro)         :  30 veces
```

De 200 puntos, el bucle externo entró **10 veces**, no 200. Esa es la razón de que tres bucles anidados no sean cúbicos.

> **Ojo con el `rng` de semilla fija.** `templateHeavy.cpp` define `mt19937 rng(0)`, así que la mezcla es **determinista** y en teoría un test adversario podría forzar el peor caso. Si te preocupa:
>
> ```
> mt19937 rng((uint32_t)chrono::steady_clock::now().time_since_epoch().count());
> ```

Casos degenerados

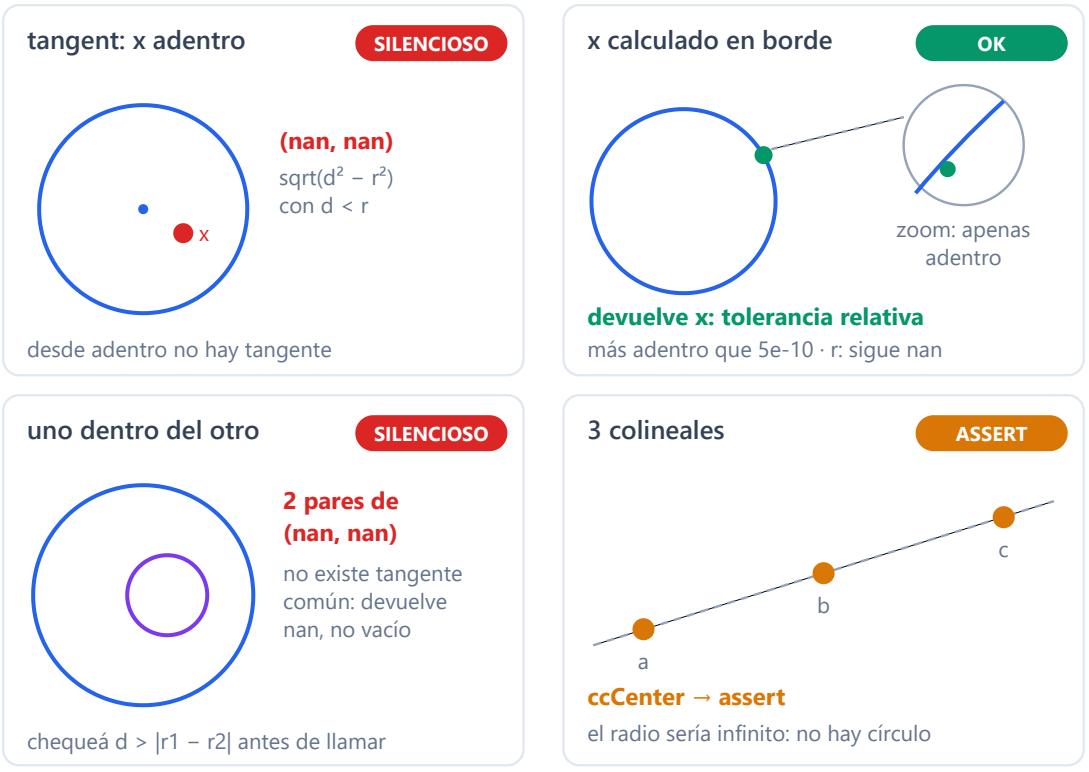


Figura 21. casos degenerados de Circles

Rojo = falla **en silencio**; naranja = `assert`; verde = el caso que parece peligroso pero está cubierto.

Todas las salidas son de correr el código.

situación	qué pasa
`isect` de dos círculos **idénticos**	`assert`

situación	qué pasa
`isect` de **concéntricos** distinto radio	`{}`
`isect` **tangente**	el mismo punto **dos veces**
`isect` con **radio 0**	rama propia: el punto, o `{}`
`isect_area` de idénticos	área del círculo (no hay assert acá)
`tangent` con el punto **adentro**	**`(nan, nan)` en silencio**
`tangent` con el punto **calculado sobre el borde**	el propio punto: tolerancia relativa, sin `nan`
`arcLength` de un arco menor a $10^{-9}r$	1% de error; con $10^{-10}r$ devuelve **0**
`external` / `internal` con uno **dentro** del otro	**dos pares de `(nan, nan)` en silencio**
`ccCenter` con 3 **colineales** o repetidos	**`assert`**
`mec` de 1 punto	radio 0
`mec` de todos iguales	radio 0
`arcLength` con `a == b`	0

> **El patrón a recordar:** `isect` y `ccCenter` **avisan** con un `assert`; las tangentes **no** — devuelven `nan` y siguen. Si tu entrada puede violar la precondición de `tangent`, `external` o `internal`, chequealo vos con `in()`.

Ejemplo completo

Dos círculos, de la relación entre ellos al área común, con las salidas intermedias:

```
Circ a = {{0,0}, 2}, b = {{3,0}, 2};

T d = abs(a.f - b.f);
cout << d << "\n";                        // 3        distancia entre centros
cout << sgn(d - (a.s + b.s)) << "\n";     // -1       d < r1+r2  => se cortan
cout << sgn(d - abs(a.s - b.s)) << "\n"; //  1       d > |r1-r2| => ninguno contiene al otro

auto p = isect(a, b);
cout << sz(p) << "\n";                    // 2
cout << p[0].f << "," << p[0].s << "\n"; // 1.5, 1.3229
cout << p[1].f << "," << p[1].s << "\n"; // 1.5, -1.3229

cout << isect_area(a, b) << "\n";         // 1.8132   area de la lente

// las tangentes externas son seguras porque ninguno contiene al otro
auto t = external(a, b);
cout << sz(t) << "\n";                    // 2 pares
// par 0: (0,-2) -> (3,-2)               radios iguales => tangentes paralelas

// PERO si uno contuviera al otro:
auto mal = external({{0,0},5}, {{0,0},1});
cout << mal[0].f.f << "\n";               // nan   <- sin aviso
```

Chequeo defensivo antes de llamar a las tangentes:

```
bool seguro = sgn(abs(x.f - y.f) - abs(x.s - y.s)) > 0;   // ninguno contiene al otro
if (seguro) { auto t = external(x, y); /* ... */ }
```

Verificación

rutina	casos	contra
`tangent`, `external`, `internal`, `ccCenter`	230.000	invariantes geométricas de contacto y tangencia
`mec`	5.900	fuerza bruta sobre todo par y toda terna
`ccCenter` (el `assert` no molesta a `mec`)	14.000	grillas degeneradas, C de 1 a 10
`isect(Circ,Circ)` con radio 0	—	caso puntual: antes `(nan,nan)`, ahora el punto real
`isect_area`	—	tangente externa, interna, idénticos, $d \approx a + b$
`tangent` con `x` calculado sobre el borde	80.000	antes 17–24% de `nan`; ahora 0, y toca exactamente en `x`
`tangent` con `x` adentro	60.000	sigue dando `nan` desde 10^{-6} del radio
`tangent` con `x` afuera, viejo vs. nuevo	99.742	**idénticas bit a bit**
`external`, `internal`	111.404	invariantes de tangencia: 0 fallas, 0 `nan`
`internal` de tangentes con centros calculados	20.000	antes 19% de `nan`; ahora 0

`CircleTangents.h`, `Circumcenter.h` y `MinEnclosingCirc.h` **no compilaban** antes del commit `57c7c7f`: a `Point.h` le faltaban `operator*(T,P)` y los compuestos `+=` / `-=`.

Los cinco compilan con las flags del `Makefile` sin warnings.

Misc

Par más cercano, MST con distancia Manhattan, y máxima cantidad de puntos colineales.

Manual de consulta: **qué hace cada método, qué le podés pasar y hasta dónde aguanta.** Los problemas donde practicar están en PROBLEMS.md.

Índice

- Cómo se arma la submission
- Choques de nombres
- `ClosestPair.h`
  - `solve`
- `ManhattanMST.h`
  - `manhattanMst`
- `MaxCollinear.h`
  - `getline`
  - `solve`
- Casos degenerados
- Ejemplo completo
- Verificación

Cómo se arma la submission

Copiás `templateHeavy.cpp` en `E.cpp` y pegás el **código** del header en la sección de template.

> **Los `#include` no van en la solución.** Ningún header de esta librería tiene uno. El campo `Requiere:` es solo un indicador de qué más pegar, y en qué orden.

header	tipo de entrada	pegá antes
`ClosestPair.h`	`vP` = `vector<pair<T,T>>`	**`Point.h`**
`ManhattanMST.h`	`vpi` = `vector<pair<int,int>>`	— nada
`MaxCollinear.h`	`vpi` = `vector<pair<int,int>>`	— nada

Los dos últimos trabajan con coordenadas **`int`** y son independientes del resto de la librería.

Choques de nombres

nombre	choca con	resultado
`solve` de `ClosestPair.h`	`solve` de `MaxCollinear.h`	conviven (sobrecarga `vP` vs `vpi`)
ambos `solve`	tu `void solve()` de `templateHeavy.cpp`	conviven (la tuya no toma argumentos)
`getline` de `MaxCollinear.h`	`std::getline`	conviven (`getline(cin, s)` sigue andando)

Los tres compilados y corridos. Pero si pegás `ClosestPair.h` y dentro de tu `solve()` escribís `solve(v)`, mirá a quién estás llamando.

`ClosestPair.h`

Requiere: `Point.h`

`solve`

```
pair<P,P> solve(vP v);
```

Los **dos puntos más cercanos** del conjunto. Devuelve el par, no la distancia: `abs(r.f - r.s)`.

Si hay puntos repetidos devuelve ese punto dos veces y corta de entrada.

Restricciones: $n \geq 2$ · **Tipo**: `db` (usa `abs`, que con `ll` trunca) · **Aguanta**: coordenadas hasta $\approx 4.3 \cdot 10^9$ con `db` = `long double` · **Complejidad**: $O(n \log n)$, barrido con `set`.



Figura 22. el barrido y la franja activa

Toma `v` **por copia**: lo ordena.

```
solve({{0,0},{5,5},{0,0}})  ->  (0,0) (0,0)     // duplicado: distancia 0
solve({{0,0},{3,4}})        ->  dist = 5
```

`ManhattanMST.h`

Independiente (no requiere `Point.h`)

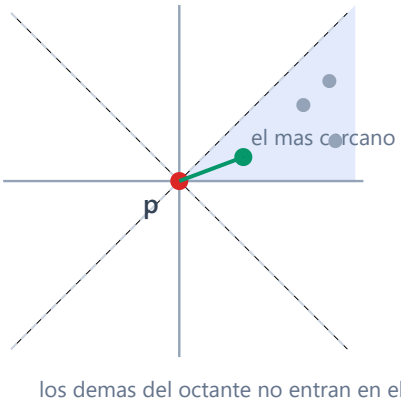
`manhattanMst`

```
V<pair<ll,pi>> manhattanMst(vpi v);
```

Devuelve **hasta $4n$ aristas** que contienen un MST del grafo completo con peso $|p_x - q_x| + |p_y - q_y|$. Formato: `{distancia, {índice_origen, índice_destino}}`.

> **No devuelve el MST, devuelve las candidatas.** Hay que correrles un MST encima.

Restricciones: coordenadas `int`; **puede haber repetidos** (generan aristas de peso 0); **no requiere ningún orden** de entrada · **Tipo**: entrada `int`, pesos `ll` · **Aguanta**: coordenadas hasta 10^9 — las diferencias llegan a $2 \cdot 10^9$ y las sumas a $4 \cdot 10^9$, por eso los pesos son `ll` y un `int` desbordaría · **Complejidad**: $O(n \log n)$.



Por que alcanzan 4N aristas

El plano se parte en 8 octantes alrededor de cada punto p.

Dentro de un octante, SOLO el vecino mas cercano puede estar en un MST: cualquier otro forma un ciclo con una arista mas corta.

Las 4 vueltas del bucle cubren los 8 octantes por simetria, y cada una aporta a lo sumo 1 arista por punto.

Figura 23. los 8 octantes y el vecino más cercano

Toma `v` **por copia**: le hace cuatro rotaciones del plano.

Las aristas pueden venir repetidas — un mismo par puede ser el vecino más cercano en más de un octante. Kruskal las descarta solo; no hace falta deduplicar.

```
manhattanMst({{0,0}})          ->  0 aristas
manhattanMst({{0,0},{0,0}})    ->  4 aristas, todas de peso 0
manhattanMst(50 puntos)        ->  157 aristas   (la cota 4n seria 200)
```

También sirve para distancia Chebyshov (L_∞): rotando el plano 45° ($x' = x + y$, $y' = x - y$), L_∞ pasa a ser L_1. Cada peso resultante es el **doble** de la distancia Chebyshov.

`MaxCollinear.h`

Independiente (no requiere `Point.h`)

`getline`

```
pair<pi,ll> getline(pi a, pi b);
```

La **clave canónica** de la recta que pasa por `a` y `b`. Dos pares de puntos de la misma recta dan la misma clave.

Restricciones: `a != b` — un `assert` lo verifica; con dos puntos iguales la dirección es $(0, 0)$, `gcd(0,0)` da 0 y se divide por cero · **Tipo**: entrada `int`, desplazamiento `ll` · **Aguanta**: con coordenadas de 10^5 el desplazamiento ya llega a 10^{10} y un `int` desbordaría; con `ll` entra hasta 10^9 · **Complejidad**: $O(\log \max |c|)$, lo domina el `gcd`.

```
getline({0,0}, {2,2})  ->  {(-1,1), 0}
getline({3,3}, {5,5})  ->  {(-1,1), 0}    // misma recta y = x, misma clave
```

> El nombre choca con `std::getline`, pero conviven.

`solve`

```
int solve(vpi p);
```

La **máxima cantidad de puntos sobre una misma recta**.

Restricciones: **no puede haber dos puntos iguales** (el `assert` de `getline` lo corta) · **Tipo**: entrada `int` · **Aguanta**: coordenadas hasta 10^9 verificado; **el límite real es** n: son $\binom{n}{2}$ inserciones en un `map`, así que $n \approx 2000$ es el techo práctico y con $n = 10^5$ es inviable · **Complejidad**: $O(n^2 \log n)$.

```
solve({})                       ->  0
solve({{0,0}})                  ->  1
solve({{0,0},{1,1},{2,2}})      ->  3    // todos alineados
solve({{0,0},{1,0},{0,1}})      ->  2    // ninguna recta tiene 3
```

Casos degenerados

Todas las salidas de esta tabla son de correr el código, no estimadas.

entrada	`ClosestPair::solve`	`manhattanMst`	`MaxCollinear::solve`
vacío	**precondición rota** ($n \geq 2$)	0 aristas	`0`
1 punto	**precondición rota**	0 aristas	`1`
2 puntos iguales	el punto dos veces, dist 0	4 aristas de peso 0	`assert` de `getline`
todos colineales	el par más cercano, normal	normal	`n`
todos iguales	el punto dos veces, dist 0	aristas de peso 0	`assert`

Ejemplo completo

MST Manhattan de 5 puntos, con las salidas intermedias:

```
vpi v = {{0,0},{3,1},{1,4},{5,5},{2,2}};

auto ed = manhattanMst(v);
cout << sz(ed) << "\n";                      // 9      (la cota 4n seria 20)
sort(all(ed));
each(e, ed) cout << e.f << ":" << e.s.f << "-" << e.s.s << " ";
// 2:1-4  2:1-4  3:4-2  4:1-0  4:4-0  5:3-2  6:3-1  6:3-4  6:3-4
//        ^^^^^ repetida, Kruskal la descarta sola

DSU D; D.init(sz(v));
ll total = 0; int usadas = 0;
each(e, ed) if (D.unite(e.s.f, e.s.s)) { total += e.f; ++usadas; }
// toma 1-4 (2), 4-2 (3), 1-0 (4), 3-2 (5)
cout << total << " " << usadas << "\n";      // 14 4   (usadas == n-1 => conexo)
```

De los 10 pares posibles nunca se generan tres: **0–2 (5)**, **0–3 (10)** y **1–2 (5)**. Los tres son el máximo **estricto** de un ciclo, así que ningún MST los usaría:

ausente	ciclo	pesos
0–2 (5)	0–4–2–0	4, 3, **5**
0–3 (10)	0–4–2–3–0	4, 3, 5, **10**
1–2 (5)	1–4–2–1	2, 3, **5**

Chebyshov con el mismo header:

```
vpi w; each(p, v) w.pb({p.f + p.s, p.f - p.s});   // rota 45 grados
auto ed2 = manhattanMst(w);                       // cada peso es el DOBLE
```

Verificación

rutina	casos	contra
`manhattanMst`	6.300	MST del grafo completo $O(n^2)$, C hasta 10^9
`getline`, `solve` (MaxCollinear)	6.000	fuerza bruta en `__int128`, C hasta 10^9
`solve` (ClosestPair)	5.500	fuerza bruta $O(n^2)$, con duplicados y colineales

Los tres compilan con las flags del `Makefile` sin warnings.

`ManhattanMST.h` **no compilaba** antes del commit `57c7c7f` (incluía `"DSU (7.6).h"`, una ruta inexistente), y `MaxCollinear.h` **crasheaba** con duplicados y desbordaba su clave con coordenadas de 10^5.